Quiz 11-1

- B-tree can return early at an internal node; B+-tree always descends to a leaf.
- So why does virtually every database use B+-trees?

힌트 보기

개념 정리 · B+-tree의 구조

- 실데이터(레코드 포인터)는 **leaf에만** 두고 내부 노드는 라우팅용 키만 담는다. 모든 leaf는 **연결리스트**로 이어져 있다.
- 데이터베이스의 비용 단위는 메모리 비교가 아니라 **디스크 I/O**다. 내부 노드에 키가 더 많이 들어가면 fan-out이 커진다.

강의 슬라이드 Ch11 p09 Quiz

정답 보기

풀이

① **낮은 트리 높이 → 디스크 I/O 감소.** 내부 노드가 데이터 대신 키만 담으므로 한 노드에 키를 더 많이 넣을 수 있어 fan-out이 커지고 높이가 낮아진다.

> 높이 1 = 디스크 접근 1번이라, I/O가 곧 성능이다.

② **효율적인 range query와 순차 스캔.** 모든 값이 leaf에 정렬되어 있고 leaf끼리 연결리스트로 이어져 있어 범위 탐색이 시작점 탐색 후 순차 스캔 $O(k)$로 끝난다.

> B-tree는 값이 내부 노드에도 흩어져 있어 범위 스캔이 비효율적이다.

③ **균일하고 예측 가능한 성능.** 모든 검색이 정확히 같은 깊이(leaf)에서 끝난다.

> B-tree의 내부 조기 종료는 운 좋을 때의 작은 이득일 뿐, 위 장점들에 압도된다.

최종 답 **fan-out↑(높이↓·I/O↓) + leaf 연결로 range query 효율 + 균일한 탐색 깊이**

한 줄 정리 · B-tree의 조기 종료는 best-case의 작은 이득, B+-tree의 **낮은 높이·순차 스캔·균일 성능**이 압도한다.